NATIONAL UNIVERSITY
of Computer & Emerging Sciences

PROJECT REPORT

SECUREAUTH - SECURE NETWORK-BASED USER AUTHENTICATION SYSTEM USING SOCKET PROGRAMMING

Submitted by

Syed Muhammad Muzammil 24k-2000

Syed Muhammad Ahsan 24k-2041

SecureAuth

NATIONAL UNIVERSITY of Computer & Emerging Sciences

Table of Contents

SecureAuth - Secure Network-Based User Authentication System using Socket Programming

1. Motivation

In modern distributed computing environments, secure identification of users across a network is one of the most important problems in computer science. Traditional password-based systems that transmit credentials directly over a network are vulnerable to eavesdropping, replay attacks, credential sniffing, brute-force attempts, and session abuse. This project was selected to demonstrate how networking concepts and security principles can be combined into a complete working authentication system.

SecureAuth connects the theory taught in Computer Networks with practical implementation. It uses TCP socket programming, client-server communication, nonce-based challenge-response authentication, HMAC proof verification, email OTP verification, TOTP multi-factor authentication, session management, rate limiting, account lockout, and audit logging. The result is not just a login page, but a layered authentication pipeline that shows how real systems defend against common network attacks.

2. Overview

SecureAuth is a secure network-based user authentication system built using Python socket programming, FastAPI, SQLite, and a React + TypeScript frontend. Users can register, verify their email address, log in using challenge-response authentication, complete QR/TOTP multi-factor authentication, and access a session-based user dashboard. Administrators can log in through a separate admin flow and monitor users, sessions, audit logs, locked accounts, and account status.

- Python socket-based authentication logic using structured JSON action messages.
- FastAPI backend that exposes frontend API routes and administrative endpoints.
- SQLite database for users, admins, sessions, OTPs, nonces, MFA state, and audit logs.
- React + TypeScript frontend for user, admin, and dashboard workflows.
- SMTP-based email OTP support using Brevo or Gmail app password configuration.
- Render deployment with a static frontend and Python web service backend.

3. Significance of the Project

The project has academic and practical significance because it implements a complete authentication lifecycle instead of only demonstrating a single concept. It applies TCP sockets, protocol design, client-server architecture, replay attack prevention, session lifecycle management, rate limiting, and monitoring in one integrated system.

Area	Significance
Academic Value	Applies Computer Networks concepts such as TCP sockets, JSON message protocols, client-server design, and secure communication.
Security Value	Combines email OTP, HMAC challenge-response, TOTP MFA, bcrypt password hashing, session expiry, lockout, and audit logging.
Practical Value	Provides working user and admin dashboards that can be demonstrated locally and deployed on Render.
Network Value	Shows how authentication events travel between frontend, REST API, socket-auth handlers, SMTP provider, database, and dashboards.

4. Description of the Project

The system is organized into a frontend, a FastAPI backend, internal socket-auth logic, and a SQLite database. In local development, the socket server can run separately to demonstrate TCP socket communication. In Render deployment, FastAPI calls the socket authentication handlers internally so that the web service does not require a public TCP socket port.

4.1 Main Components

Component	Technology	Role
Frontend	React, TypeScript, Vite, shadcn/ui	Provides register, login, MFA, user dashboard, and admin dashboard pages.
FastAPI Backend	Python, FastAPI, Uvicorn	Exposes REST API routes for frontend requests and admin/user operations.
Socket Auth Logic	Python socket handlers, JSON actions	Processes registration, OTP verification, login, HMAC proof, MFA, and logout actions.
Database	SQLite	Stores users, admins, sessions, OTP records, nonces, audit logs, and account status fields.
Email Provider	Brevo or Gmail SMTP	Delivers OTP codes for registration and email change verification.
Authenticator App	TOTP compatible apps	Generates MFA codes after QR setup.

4.2 Authentication Flow

1. Registration begins when the user submits profile details and a strong password.
2. The backend validates the request and generates a single-use email OTP.
3. The user verifies the OTP, after which the account is created with a hashed password and MFA secret.
4. During login, the server verifies the password and generates a nonce challenge.
5. The frontend computes an HMAC response using the password-derived key and nonce.
6. The backend verifies the nonce, expiry, single-use status, and HMAC proof.
7. The user completes TOTP MFA and receives a session token with expiry.
8. Admin login follows a separate credentials plus MFA flow before opening the admin dashboard.

5. Background of the Project

The project is based on established networking and security concepts. TCP provides reliable, ordered, connection-oriented communication for the authentication protocol. HMAC provides proof of knowledge without sending plaintext passwords. TOTP adds a second factor based on time-synchronized one-time codes. bcrypt protects stored passwords against offline cracking. Rate limiting, account lockout, session expiry, and audit logging follow common authentication security guidelines.

- TCP Socket Programming: reliable client-server communication over the transport layer.
- HMAC-SHA256: challenge-response verification using a server-generated nonce.
- TOTP MFA: time-based one-time passwords as described in RFC 6238.
- bcrypt: salted password hashing with resistance against precomputed attacks.
- OWASP Authentication Guidelines: lockout, rate limiting, session expiry, and audit logging practices.

6. Project Category

This project falls under the Product-Based category. It is a complete software product with a frontend, backend, database, authentication protocol, administrative dashboard, and deployment configuration. It can be run locally for demonstration and deployed online using Render.

7. Features, Scope, and Modules

No.	Feature	Description
1	Challenge-Response Authentication	Server creates a nonce and the frontend returns an HMAC proof. Passwords are not transmitted as plaintext during proof verification.
2	Email OTP Verification	Registration and email-change flows use single-use OTP codes sent through SMTP and stored with expiry.
3	QR/TOTP MFA	Users and admins verify a TOTP code from an authenticator app before receiving a session.
4	Secure Password Hashing	Passwords are hashed using bcrypt before storage.
5	Session Management	Session tokens are generated securely, stored with expiry, and can be revoked.
6	Rate Limiting	Login and registration attempts are throttled to reduce brute-force abuse.
7	Account Lockout	Repeated failed attempts lock an account temporarily.
8	Admin Dashboard	Admin can view users, sessions, logs, locked accounts, and settings.
9	Soft Delete and Suspension	Admins can suspend or soft-delete users without immediately removing all records.
10	Audit Logging	Security-relevant actions are stored for monitoring and review.

8. Project Planning

Week	Activities	Responsible
Week 1	Designed JSON-over-TCP protocol, database schema, password hashing, and project structure.	Both members
Week 2	Implemented socket authentication server, registration, login, and challenge-response mechanism.	Syed Muhammad Ahsan
Week 3	Implemented email OTP, TOTP MFA, session tokens, account lockout, rate limiting, and audit logs.	Both members
Week 4	Developed FastAPI routes, React frontend pages, admin dashboard, user dashboard, and integration.	Syed Muhammad Muzammil
Week 5	Performed testing, deployment setup on Render, documentation, diagrams, and final report preparation.	Both members

9. Project Feasibility

9.1 Technical Feasibility

The system is technically feasible using mature open-source technologies. Python provides socket programming support, FastAPI provides a stable REST API layer, SQLite is suitable for a single-instance demo database, and React with TypeScript provides a reliable frontend platform. The security libraries used, such as bcrypt and pyotp, are well-supported.

9.2 Economic Feasibility

The project uses free and open-source tools. Local execution requires no paid infrastructure. Online deployment can be demonstrated on Render free tier, and OTP email can be configured with a free or low-cost SMTP service such as Brevo or Gmail app passwords.

9.3 Schedule Feasibility

The work was completed within the planned project timeline. The modular structure allowed socket logic, backend APIs, frontend pages, and documentation to be implemented and tested in stages.

10. Hardware and Software Requirements

Category	Requirement
Hardware	Standard laptop or desktop computer with at least 8 GB RAM and internet access.
Backend Runtime	Python 3.11 or newer.
Frontend Runtime	Node.js 20.19 or newer, or Node.js 22.13 or newer.
Backend Libraries	FastAPI, Uvicorn, Pydantic, bcrypt, pyotp, qrcode, python-dotenv.
Frontend Libraries	React, TypeScript, Vite, shadcn/ui components.
Database	SQLite.
Email Service	Brevo SMTP or Gmail SMTP with app password.
Deployment	Render static site for frontend and Render Python web service for backend.

11. Implementation Details

11.1 Database Schema

Table	Purpose
users	Stores user profile data, password hash, email verification status, MFA secret, lockout, suspension, and soft-delete status.
admins	Stores administrator credentials and MFA secret.
sessions	Stores active and revoked user sessions with expiry.
admin_sessions	Stores admin session tokens separately from user sessions.
pending_email_verifications	Stores email OTP codes, purposes, expiry, and usage status.
login_nonces	Stores nonce challenges for challenge-response login.
mfa_temp_tokens	Stores temporary MFA state between login verification and TOTP verification.
audit_logs	Stores security events and admin actions.

11.2 Security Controls

- Passwords are hashed using bcrypt before storage.
- Nonces are time-limited and marked used after verification.
- HMAC proof is verified instead of trusting plaintext password transmission.
- Email OTPs are stored with purpose, expiry, and single-use flags.
- MFA temporary tokens expire and are cleared after verification.
- Session tokens expire and can be revoked.
- Admin and user sessions are separated.
- Audit logs record registration, login, MFA, session, and admin actions.

12. Testing and Validation

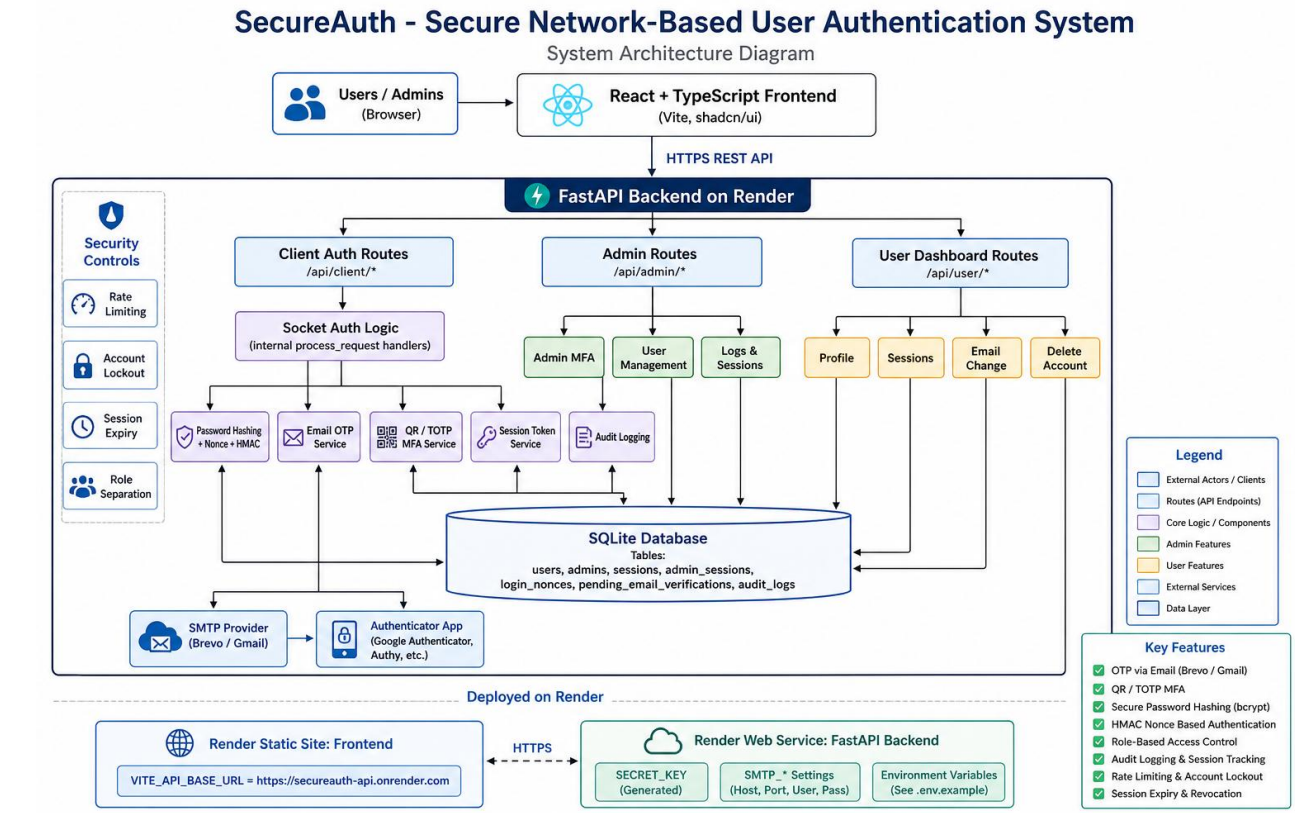
Test Case	Expected Result	Status
User registration with valid data	OTP is sent and pending verification record is created.	Passed
Invalid or expired OTP	Registration is rejected with an error.	Passed
Correct password login	Nonce challenge is generated.	Passed
Invalid HMAC proof	Login verification is rejected.	Passed
Correct TOTP MFA	Session token is created.	Passed
Repeated failed login attempts	Account lockout is applied.	Passed
Admin login and MFA	Admin dashboard opens after MFA.	Passed
Admin user management	Lock, unlock, suspend, delete, and session revoke actions are handled.	Passed
Render deployment	Frontend connects to backend through VITE_API_BASE_URL.	Passed

13. Deployment Summary

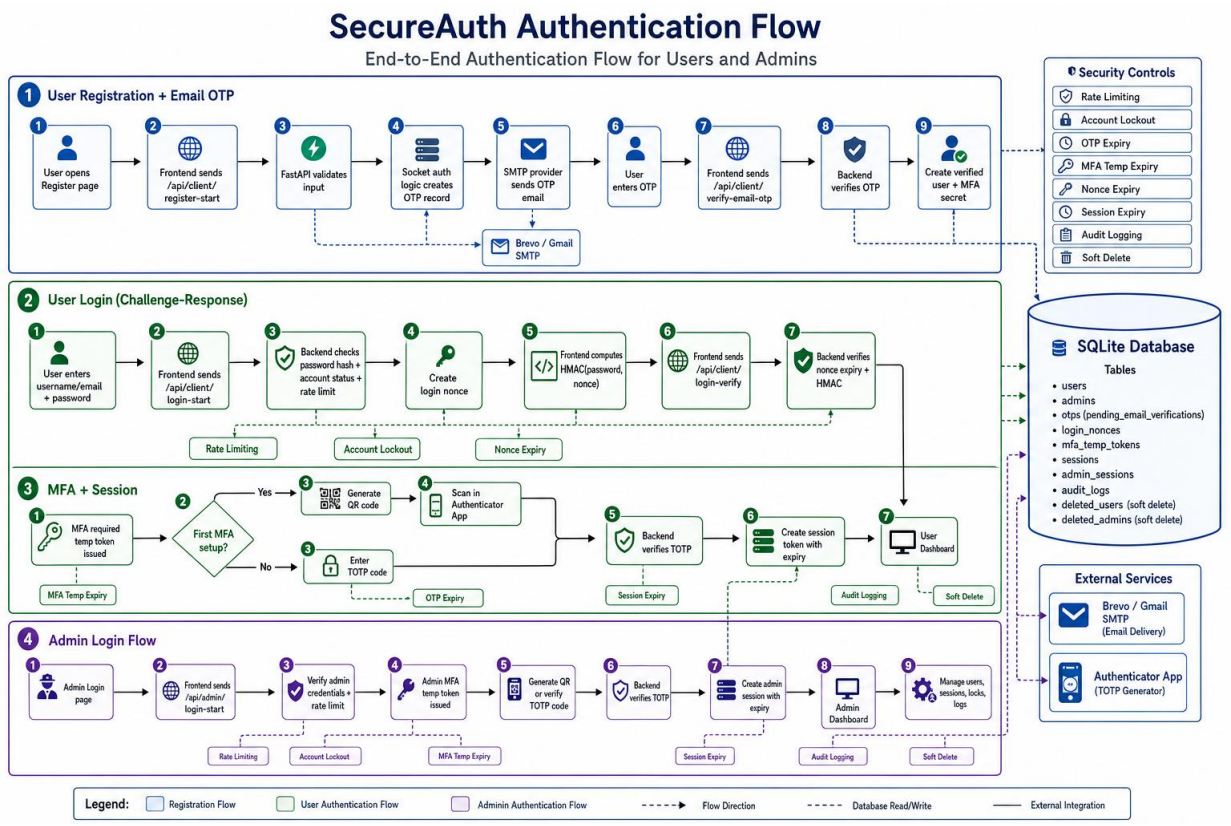
The project was prepared for deployment using Render. The frontend is deployed as a Render Static Site, while the FastAPI backend is deployed as a Render Python Web Service. The backend health endpoint is /health. Environment variables are used for SMTP credentials, secret key, database path, CORS origins, session expiry, lockout, and rate limiting configuration.

Service	Render Type	Purpose
secureauth-frontend	Static Site	Hosts the React + TypeScript frontend.
secureauth-api-v2	Python Web Service	Runs FastAPI backend and internal socket-auth handlers.
SQLite Database	File-based database	Stores demo data for the course project.
Brevo SMTP	External SMTP provider	Sends OTP emails for verification.

14. System Architecture Diagram



15. Authentication Flow Diagram



16. Demo Screenshot Placeholders

Paste final demo screenshots in the spaces below. Each screenshot should show the browser URL or application state clearly enough for evaluation.

Figure 3. Home Page / Landing Page Screenshot

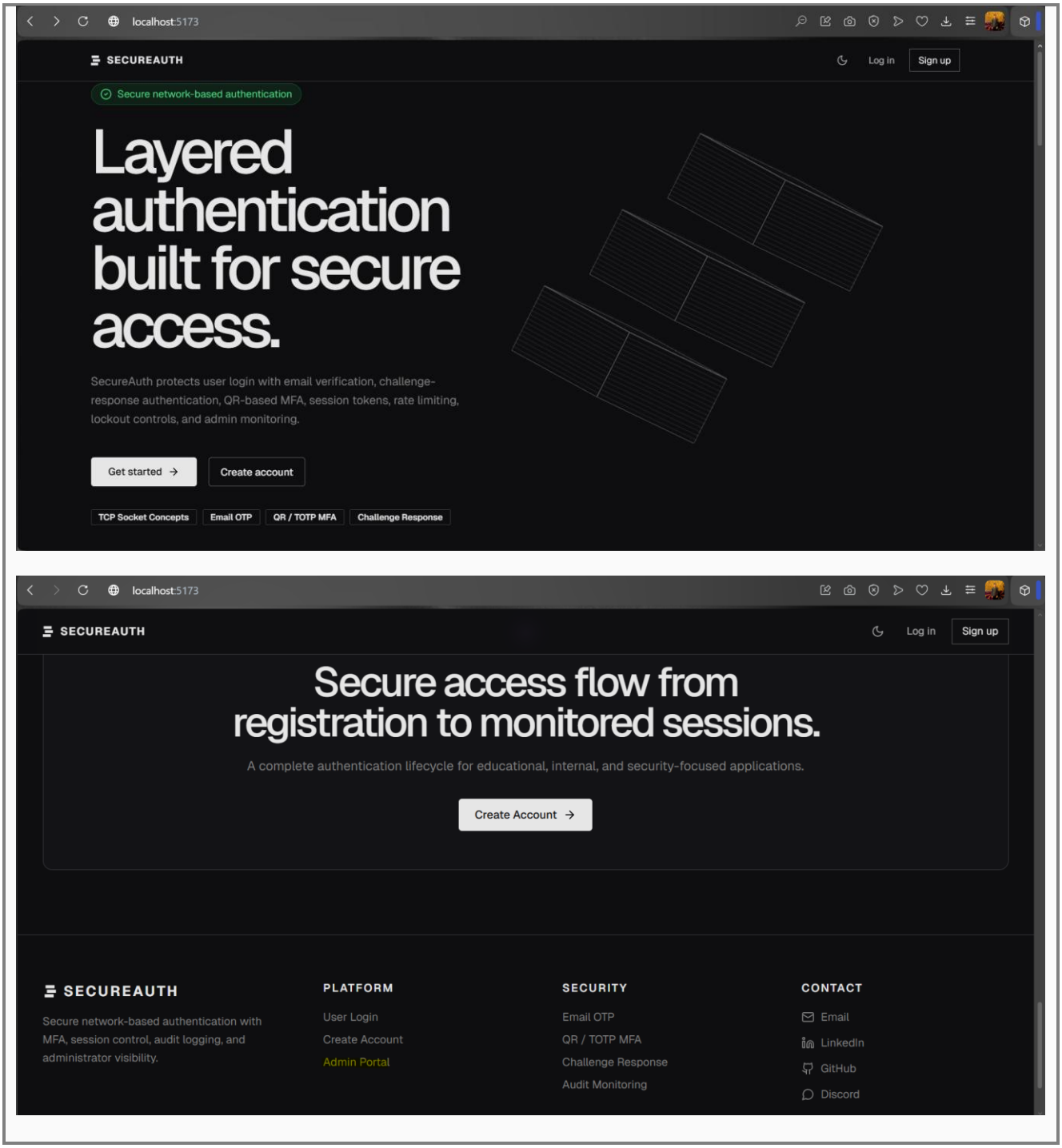


Figure 4. User Registration Form Screenshot

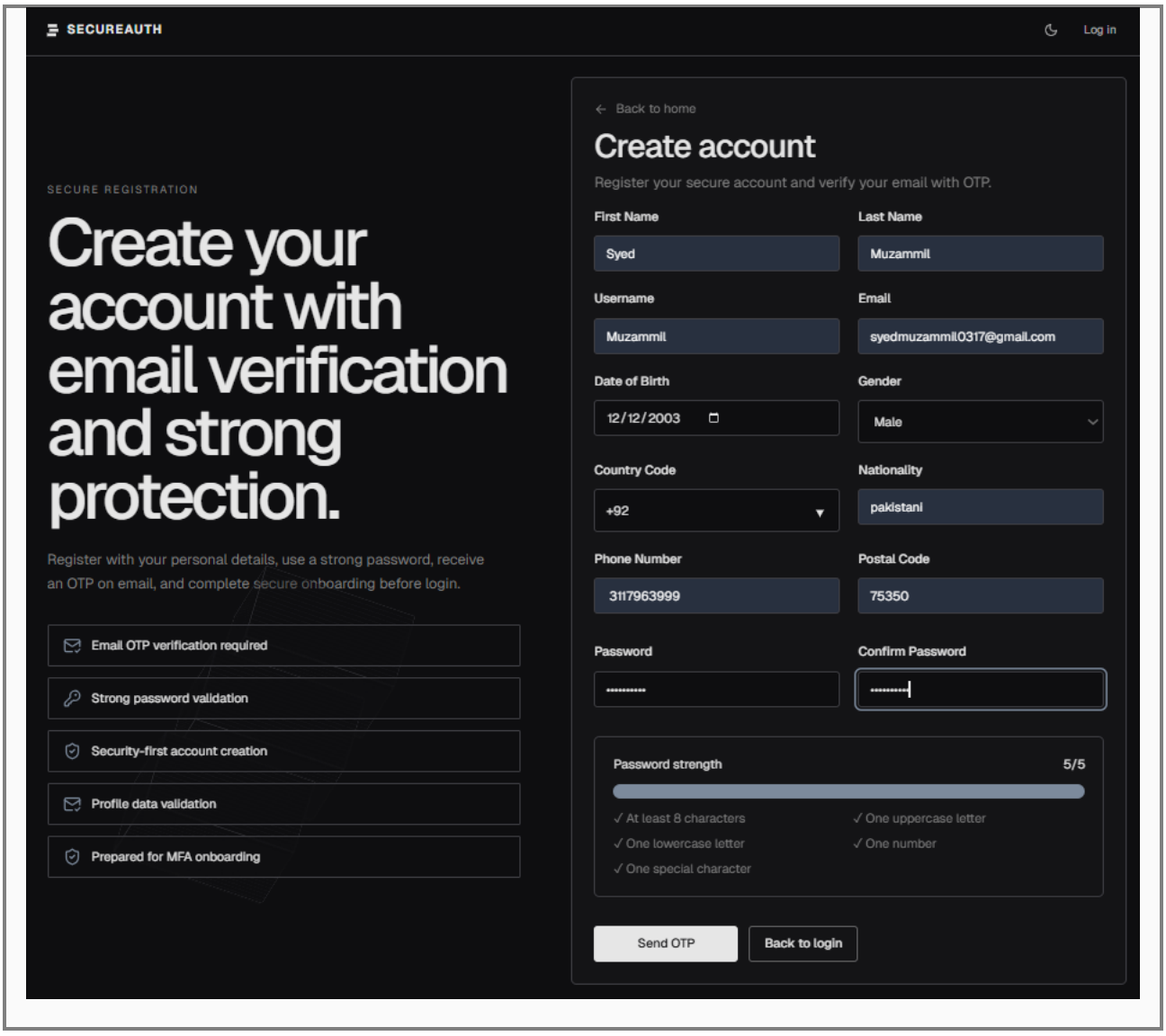


Figure 5. Email OTP Received / Verification Page Screenshot

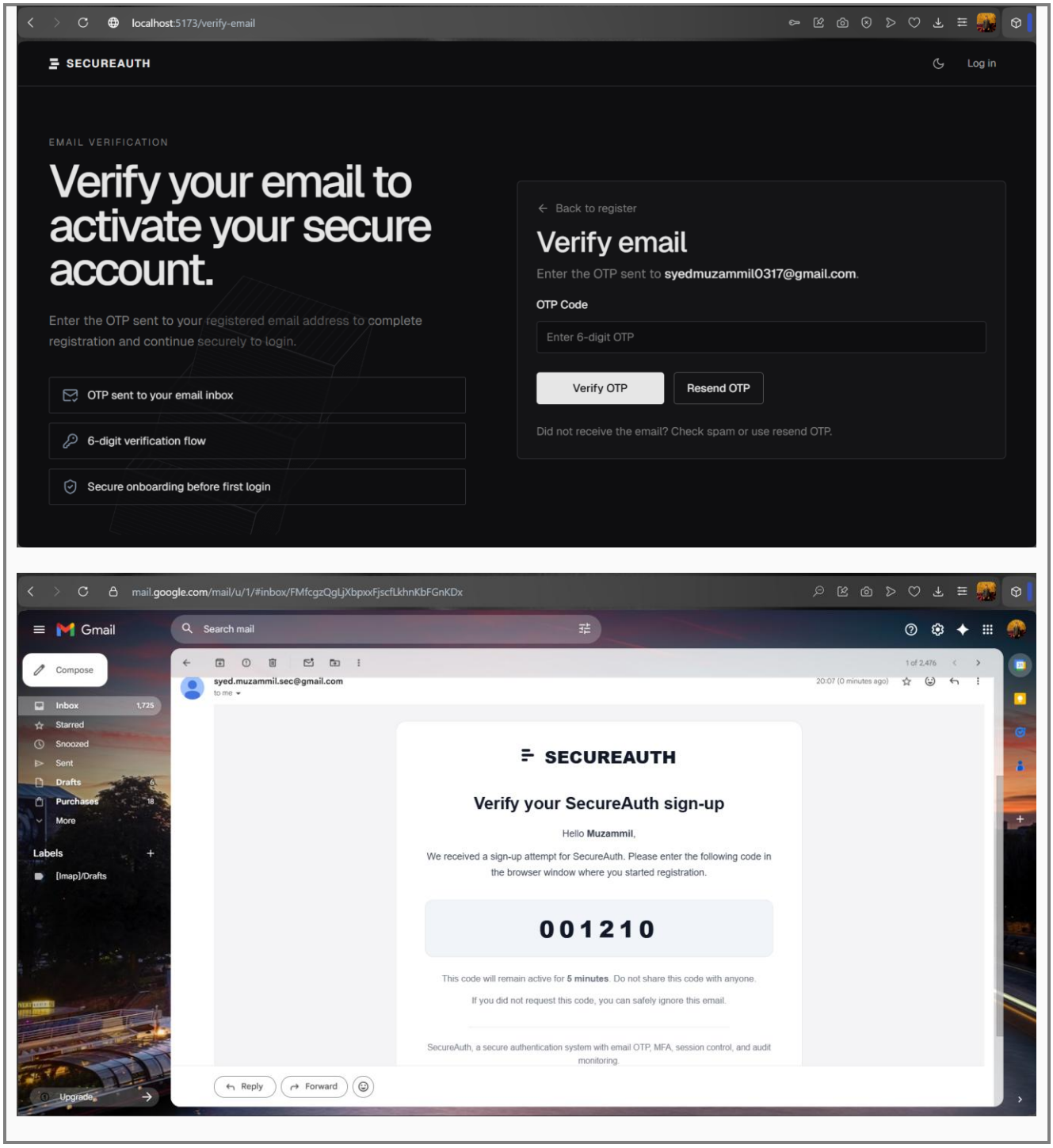


Figure 6. Login Challenge-Response Page Screenshot

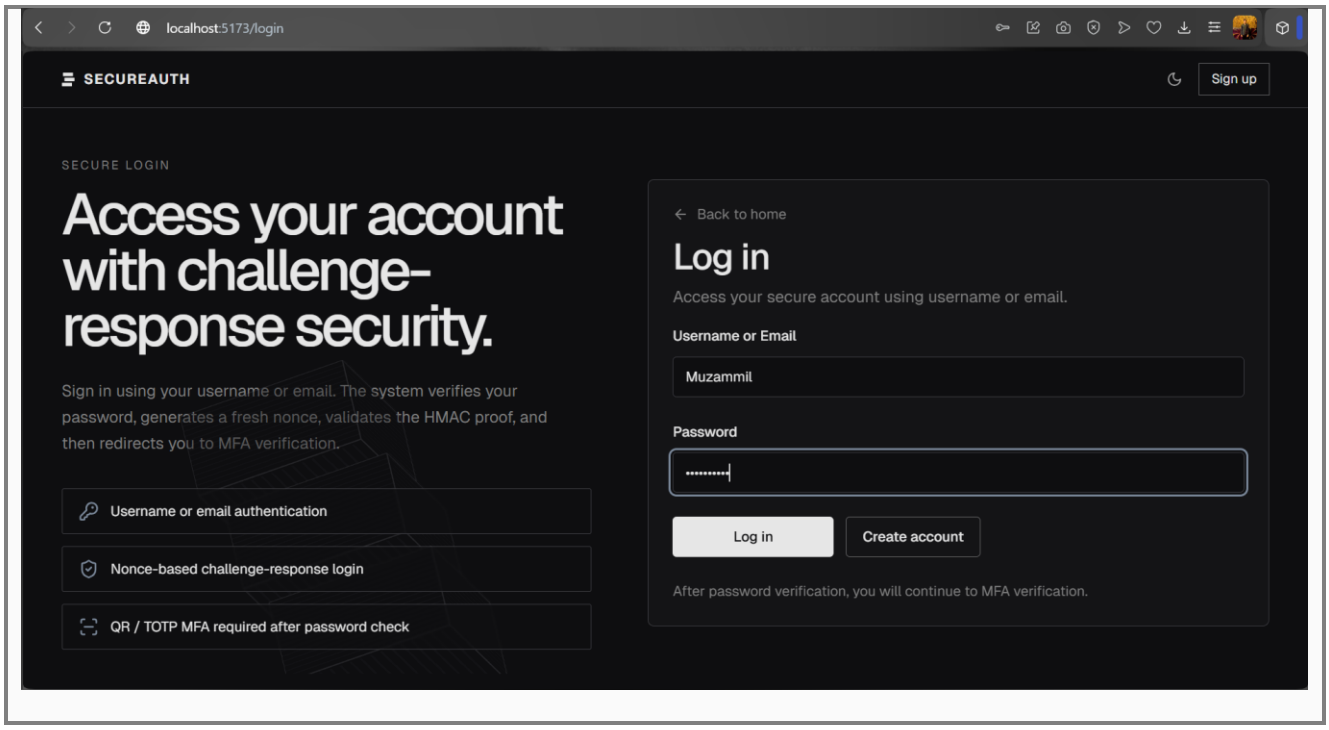
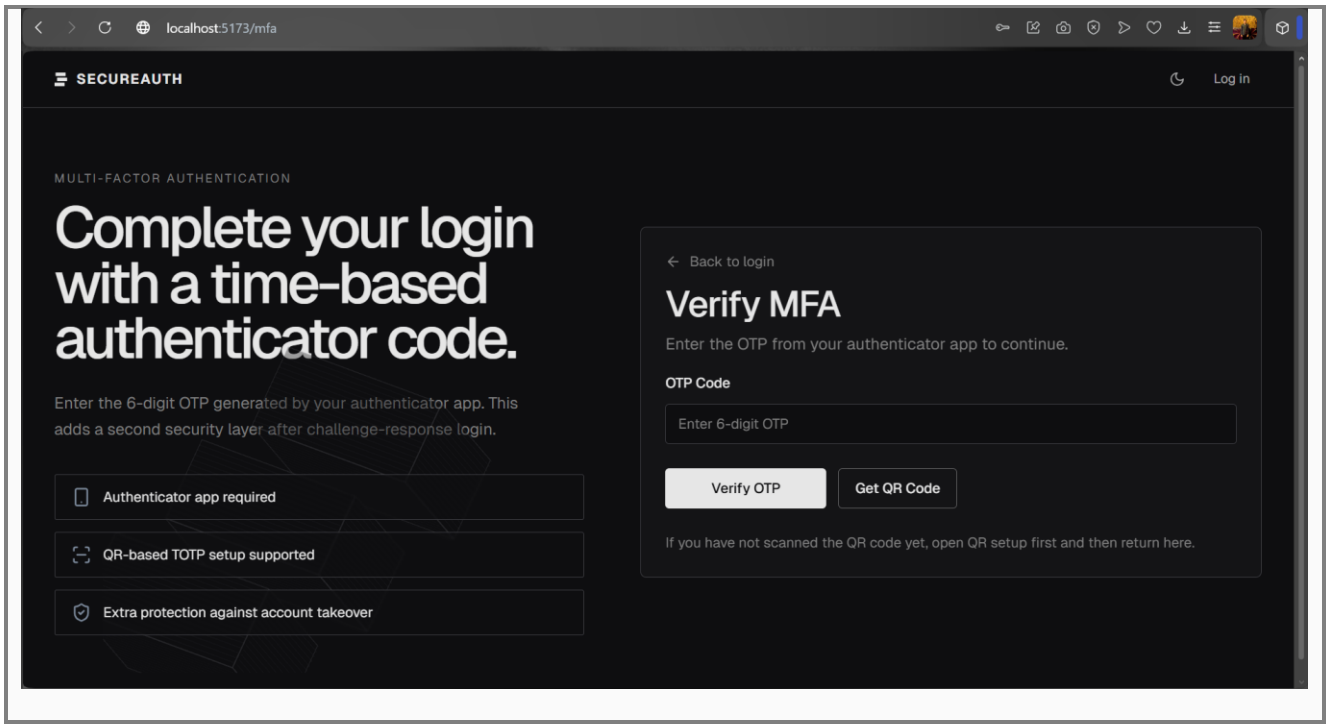


Figure 7. QR Code / Authenticator MFA Setup Screenshot



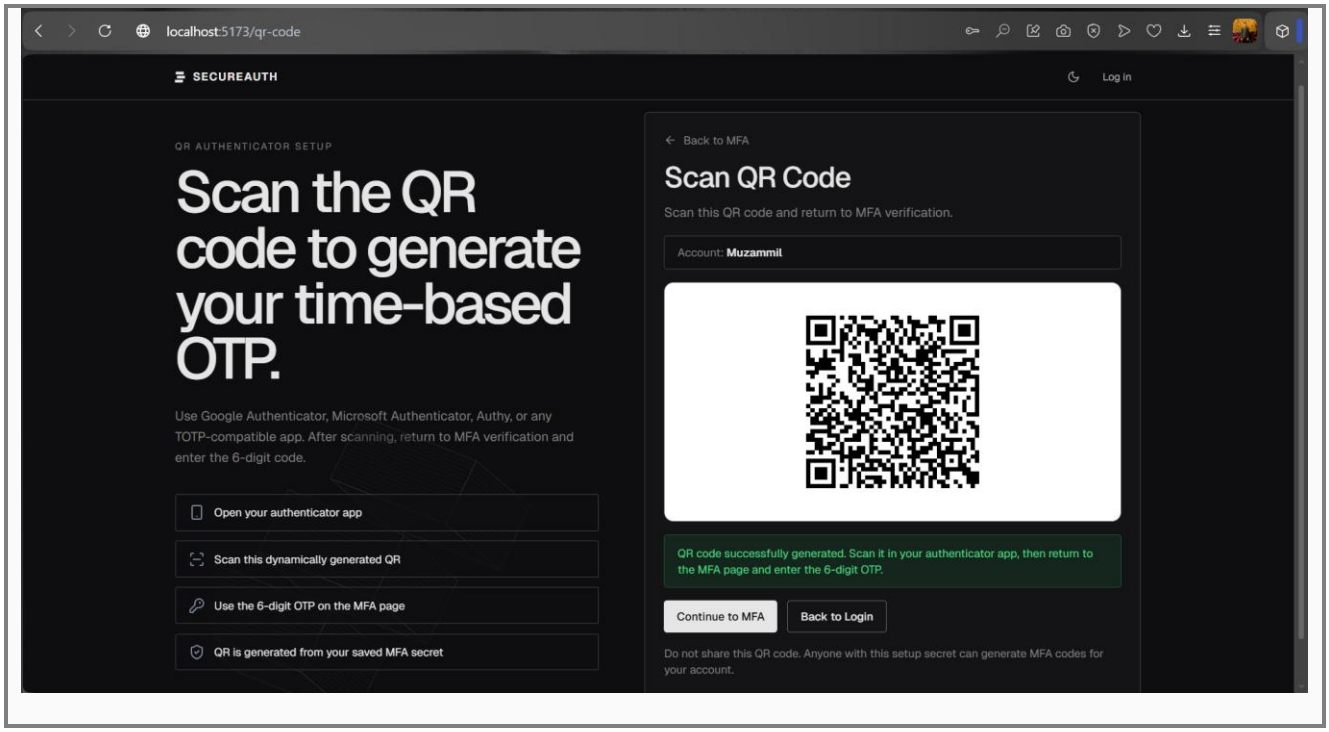
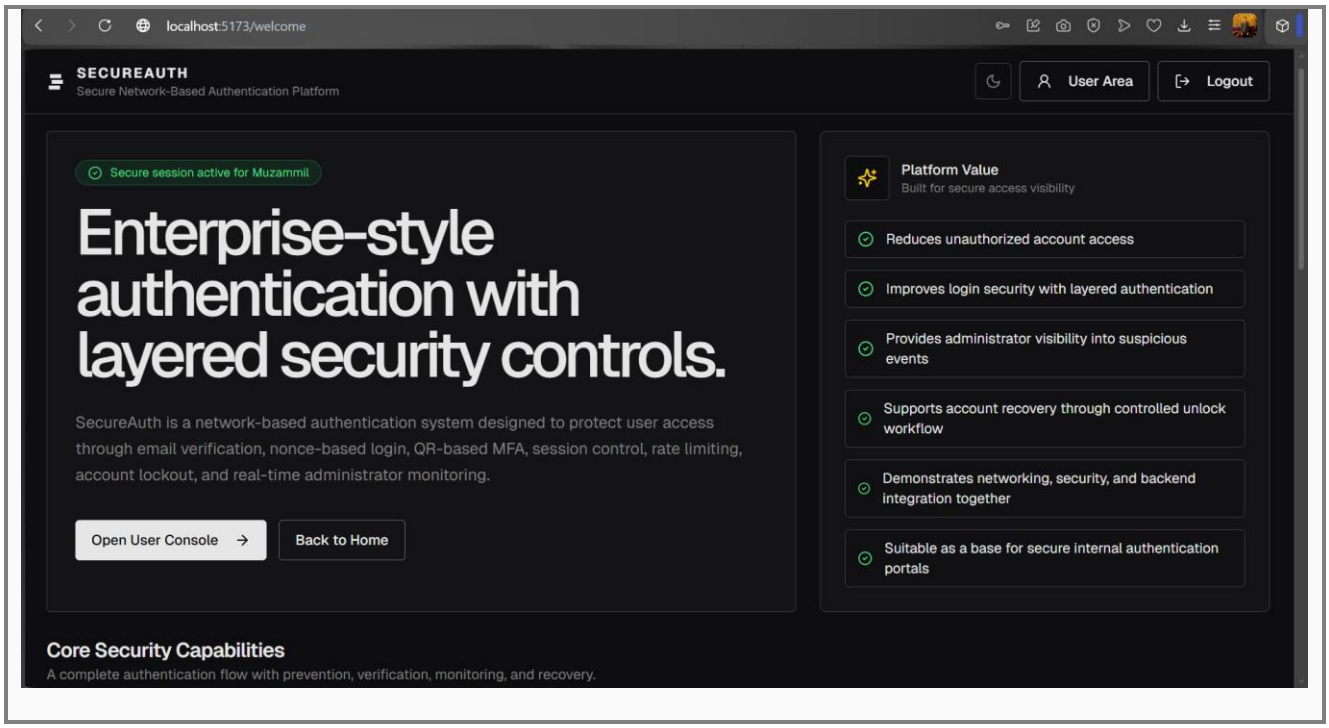
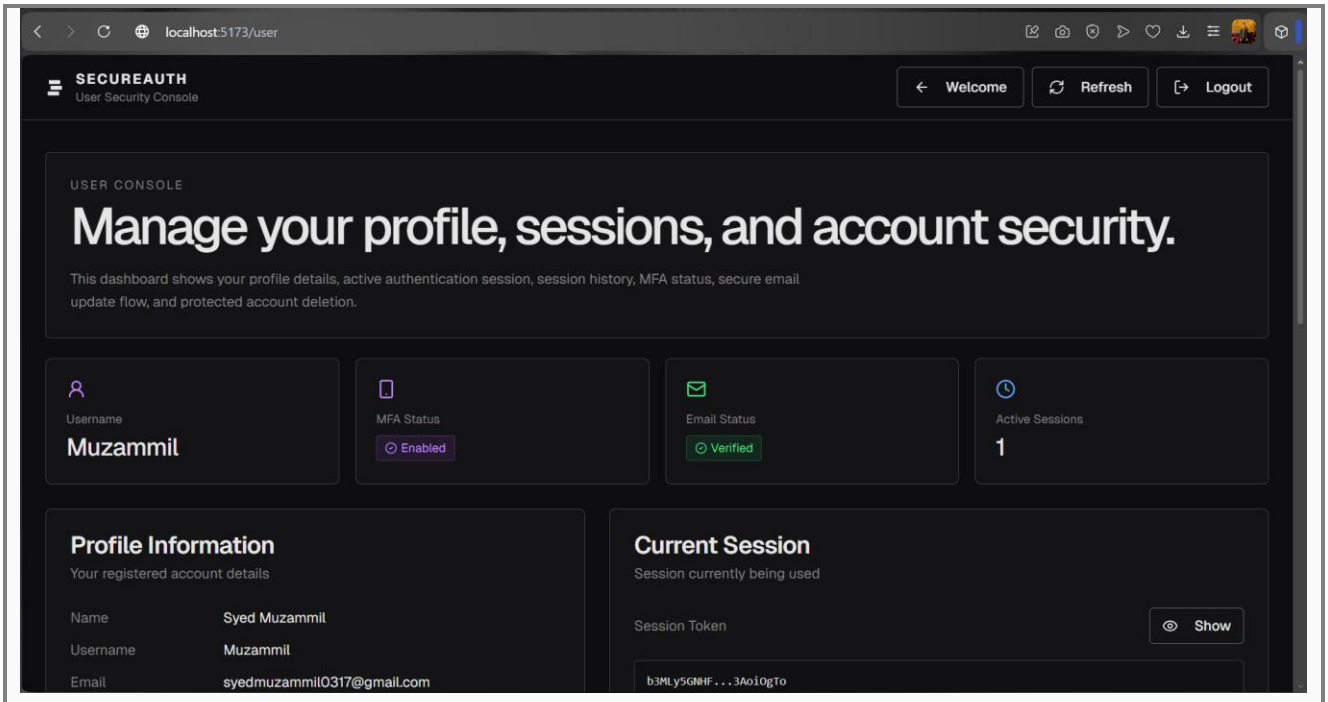


Figure 8. User Dashboard and Active Sessions Screenshot




localhost:5173/user
SECUREAUTH
User Security Console
Welcome
Refresh
Logout
USER CONSOLE
Manage your profile, sessions, and account security.
This dashboard shows your profile details, active authentication session, session history, MFA status, secure email update flow, and protected account deletion.
Username
Muzammil
MFA Status
Enabled
Email Status
Verified
Active Sessions
1
Profile Information
Your registered account details
Name
Syed Muzammil
Username
Muzammil
Email
syedmuzammil0317@gmail.com
Current Session
Session currently being used
Session Token
Show
b3MLy5GNHF...3AoiOgTo

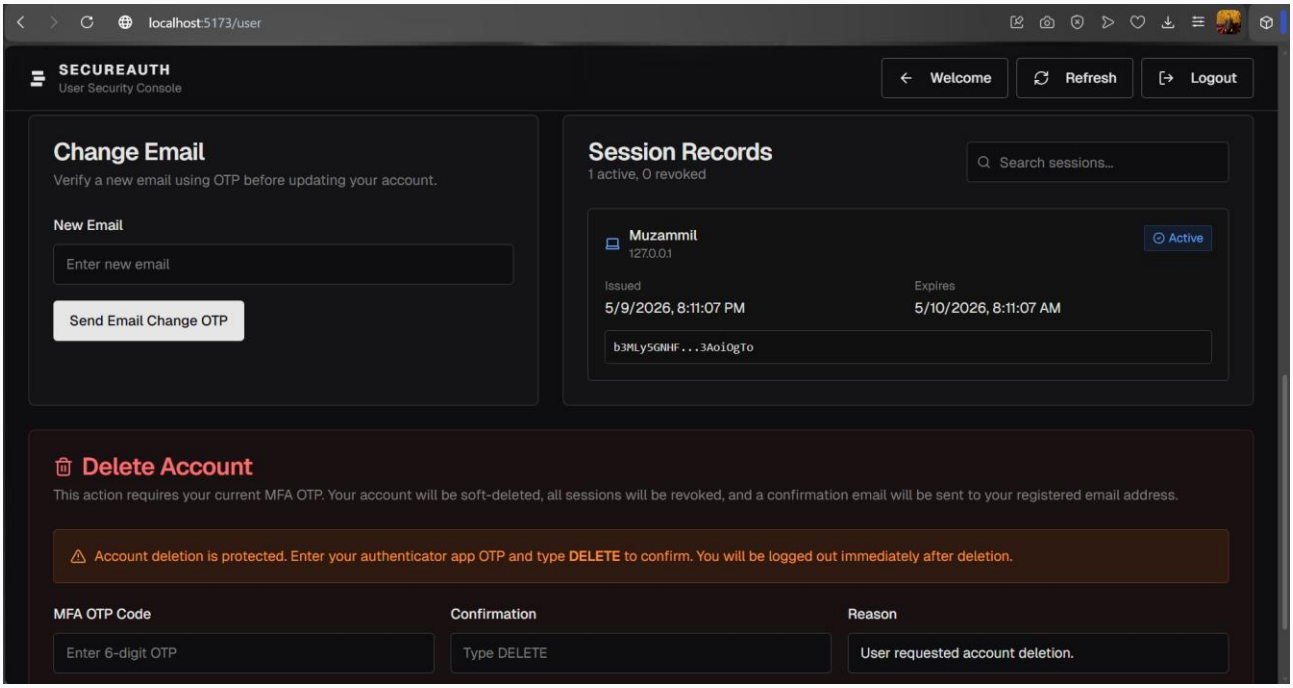
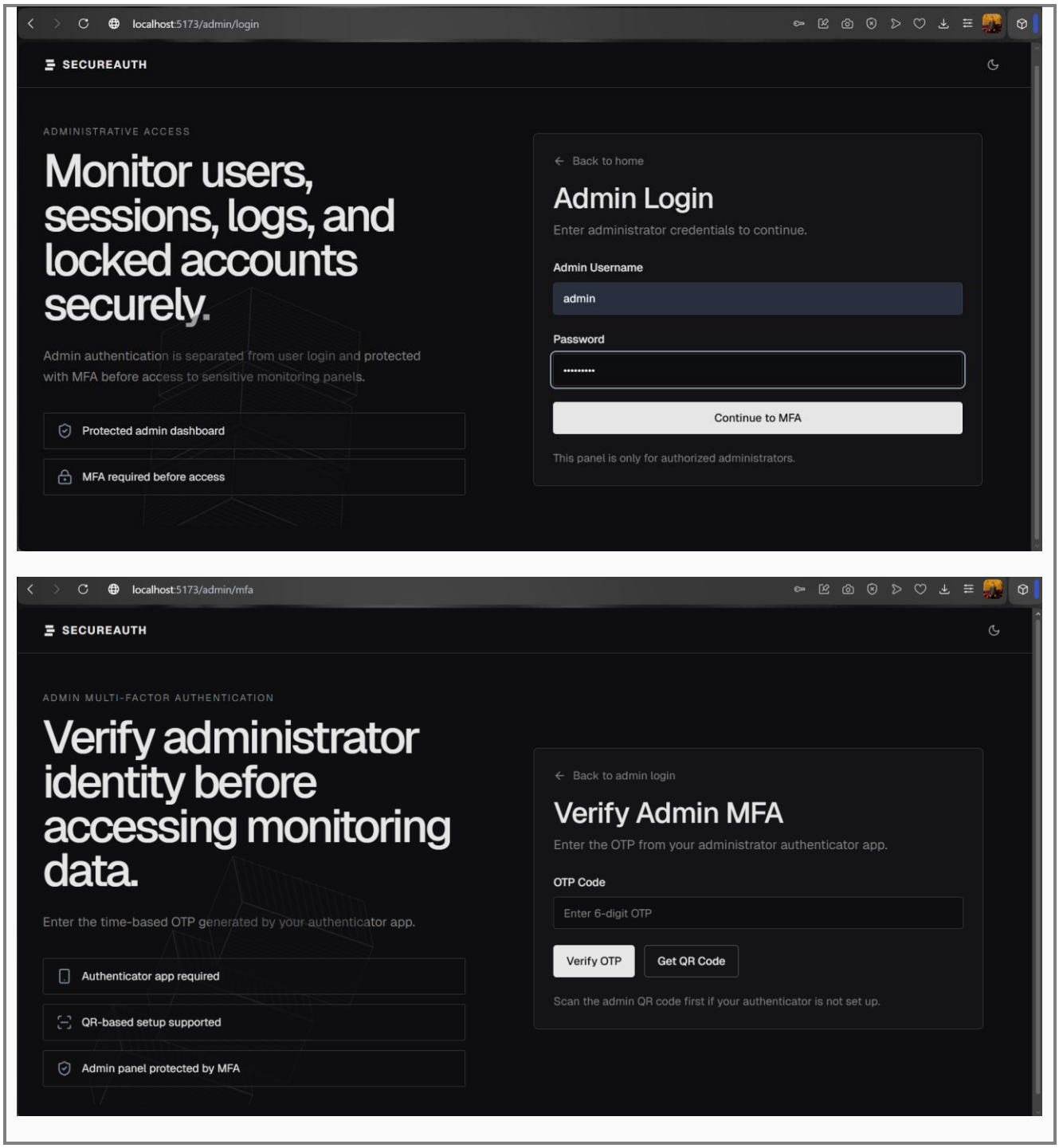

localhost:5173/user
SECUREAUTH
User Security Console
Welcome
Refresh
Logout
Change Email
Verify a new email using OTP before updating your account.
New Email
Enter new email
Send Email Change OTP
Session Records
1 active, 0 revoked
Search sessions...
Muzammil
127.0.0.1
Active
Issued
5/9/2026, 8:11:07 PM
Expires
5/10/2026, 8:11:07 AM
b3MLy5GNHF...3AoiOgTo
Delete Account
This action requires your current MFA OTP. Your account will be soft-deleted, all sessions will be revoked, and a confirmation email will be sent to your registered email address.
Account deletion is protected. Enter your authenticator app OTP and type DELETE to confirm. You will be logged out immediately after deletion.
MFA OTP Code
Enter 6-digit OTP
Confirmation
Type DELETE
Reason
User requested account deletion.

Figure 9. Admin Login and MFA Screenshot



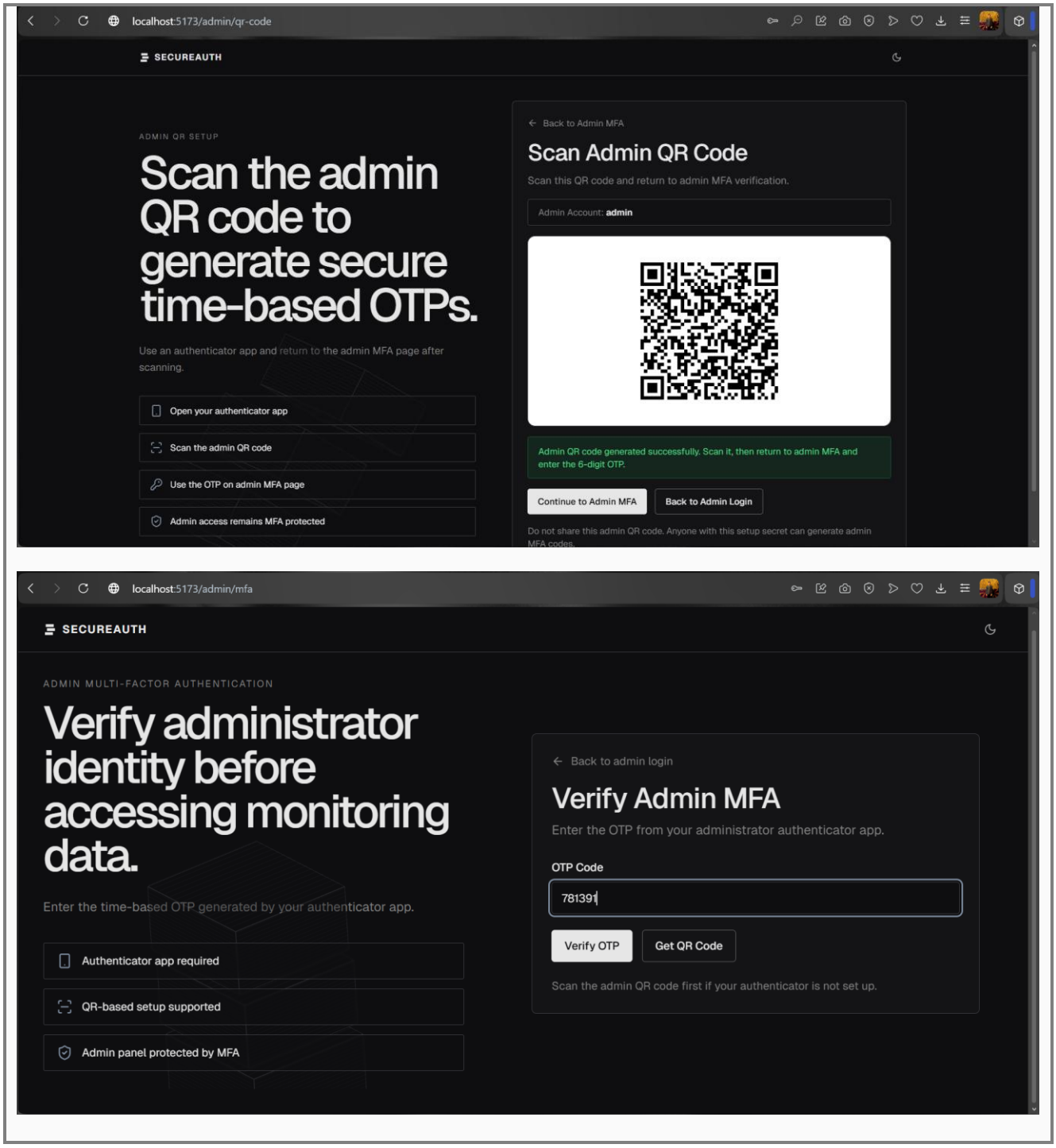

localhost:5173/admin/qr-code
SECUREAUTH
ADMIN QR SETUP
Scan the admin QR code to generate secure time-based OTPs.
Use an authenticator app and return to the admin MFA page after scanning.
Open your authenticator app
Scan the admin QR code
Use the OTP on admin MFA page
Admin access remains MFA protected
Back to Admin MFA
Scan Admin QR Code
Scan this QR code and return to admin MFA verification.
Admin Account: admin
Admin QR code generated successfully. Scan it, then return to admin MFA and enter the 6-digit OTP.
Continue to Admin MFA
Back to Admin Login
Do not share this admin QR code. Anyone with this setup secret can generate admin MFA codes.
localhost:5173/admin/mfa
SECUREAUTH
ADMIN MULTI-FACTOR AUTHENTICATION
Verify administrator identity before accessing monitoring data.
Enter the time-based OTP generated by your authenticator app.
Authenticator app required
QR-based setup supported
Admin panel protected by MFA
Back to admin login
Verify Admin MFA
Enter the OTP from your administrator authenticator app.
OTP Code
781391
Verify OTP
Get QR Code
Scan the admin QR code first if your authenticator is not set up.

Figure 10. Admin Dashboard Overview Screenshot

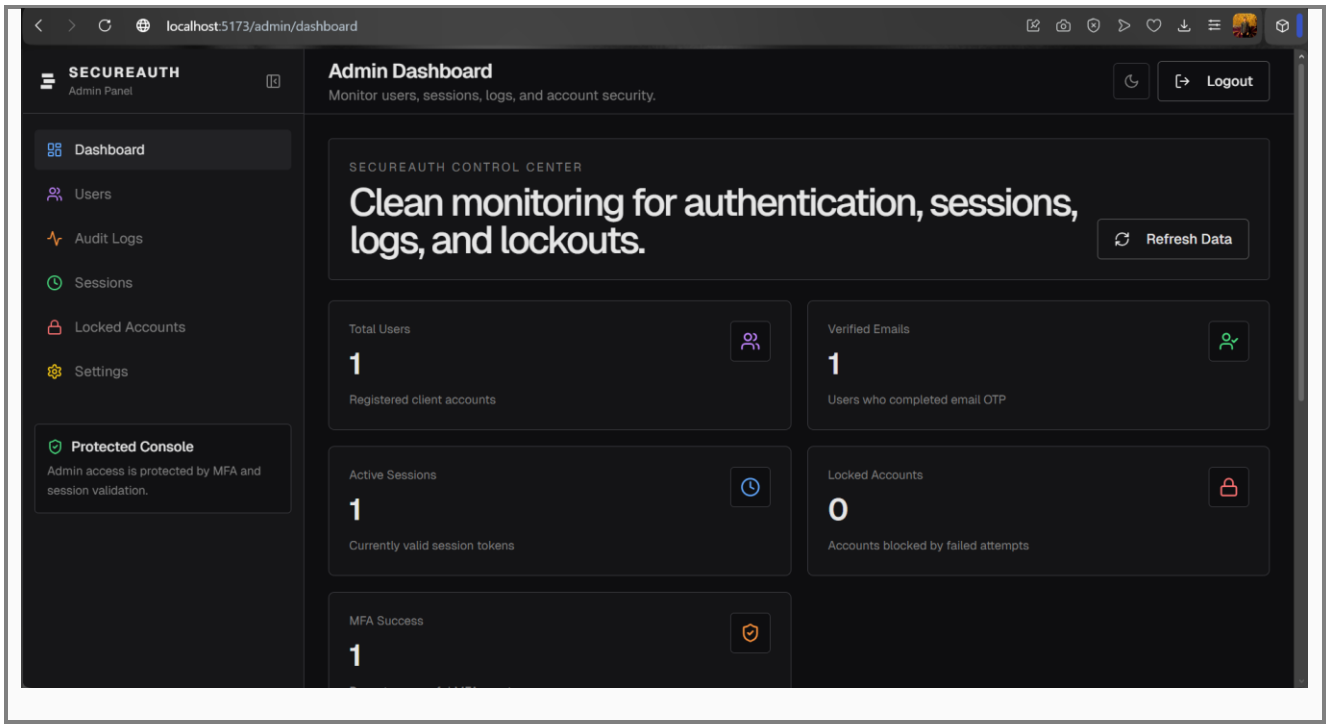


Figure 11. Admin User Management Screenshot

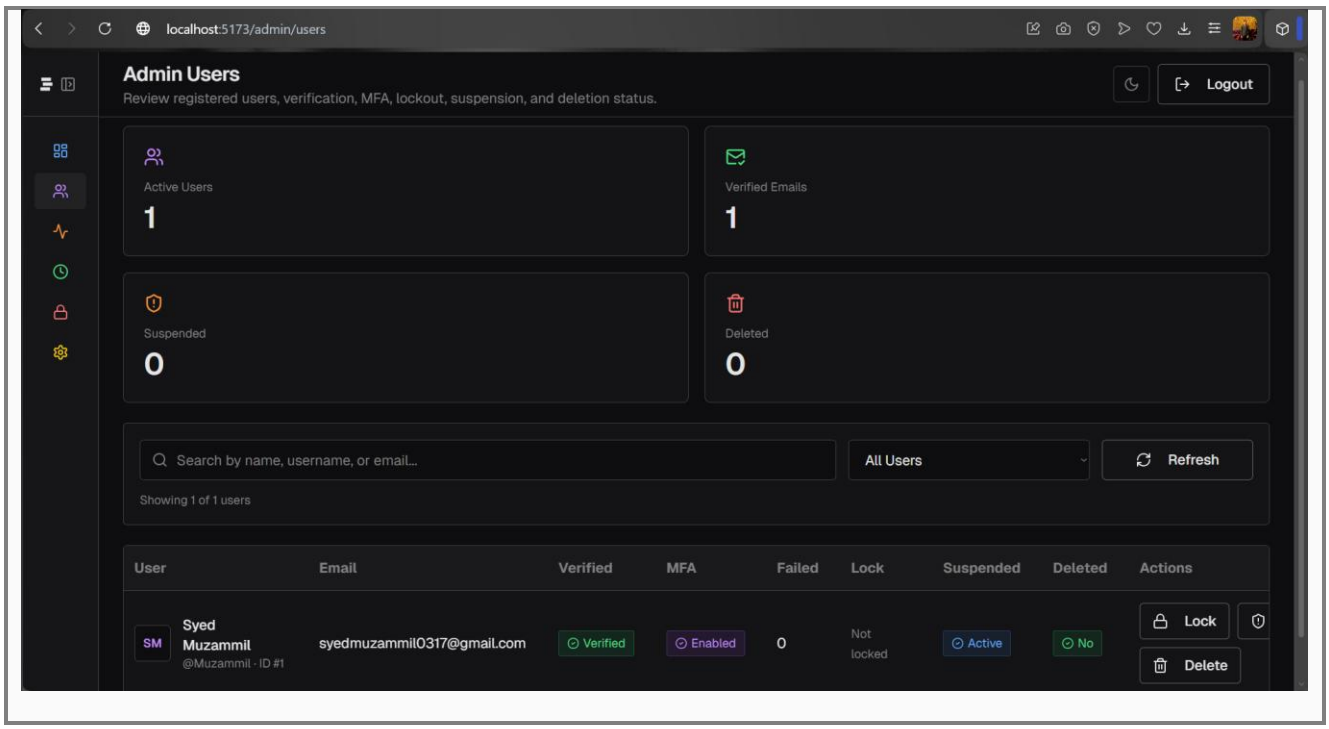


Figure 12. Audit Logs / Clear Logs Screenshot

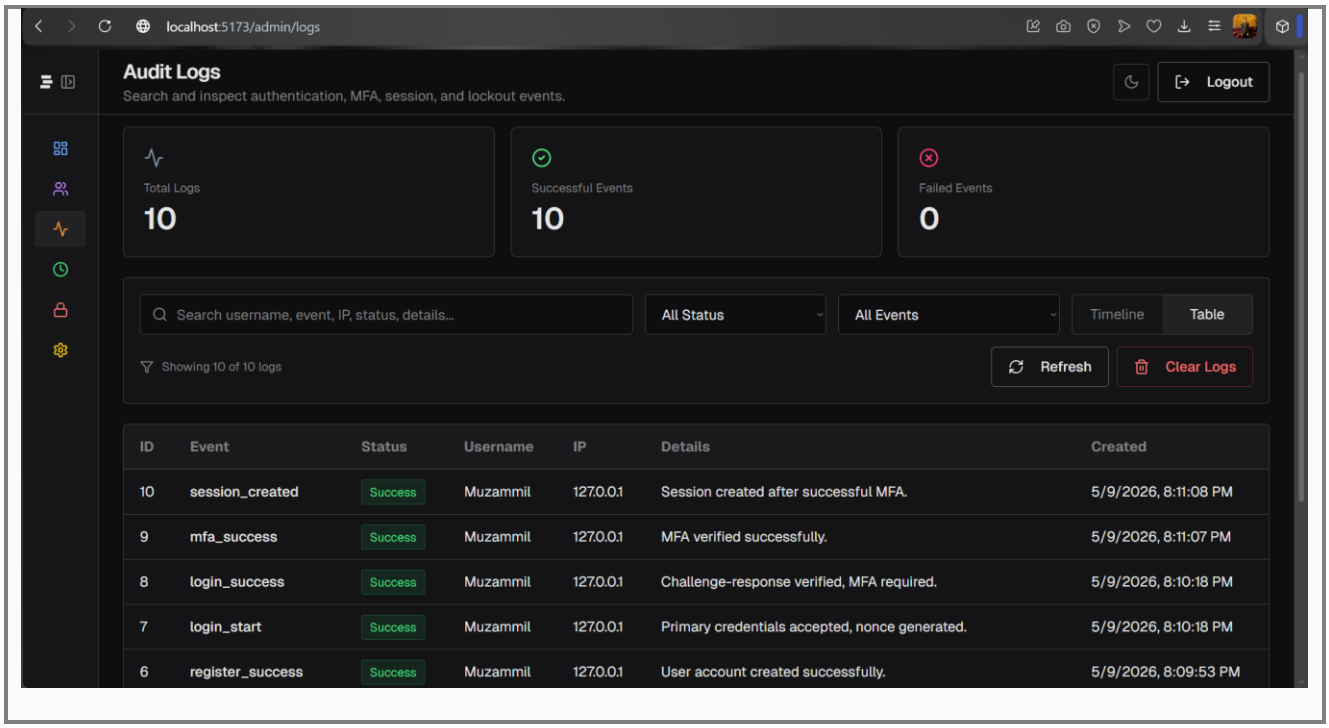


Figure 13. FastAPI Backend

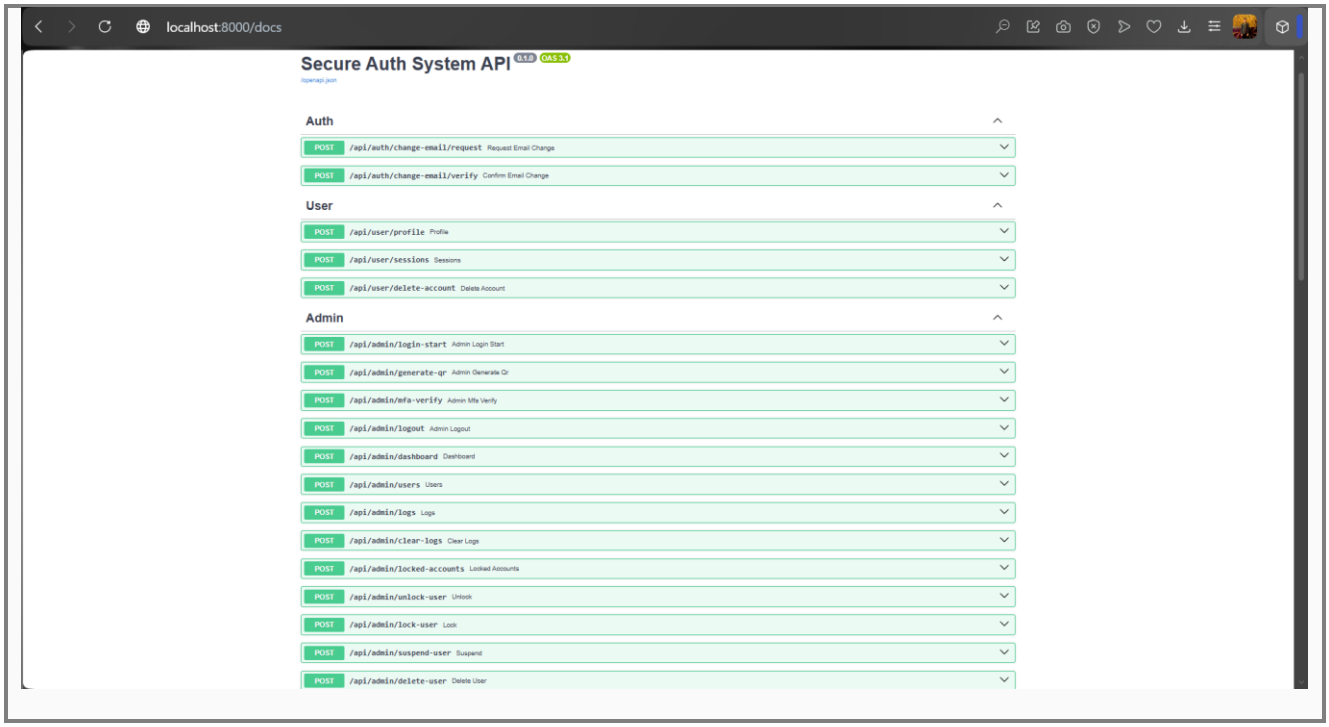
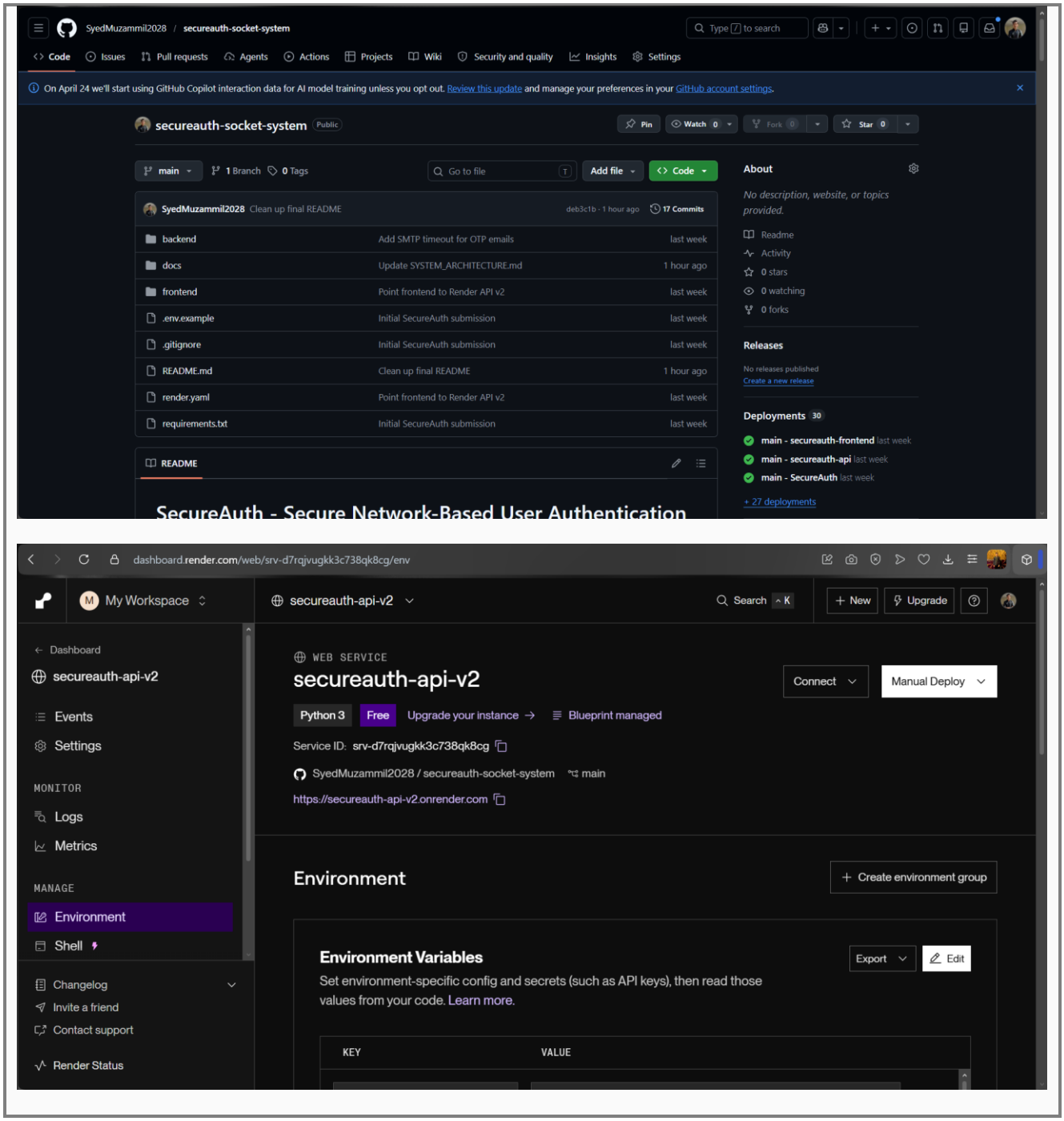
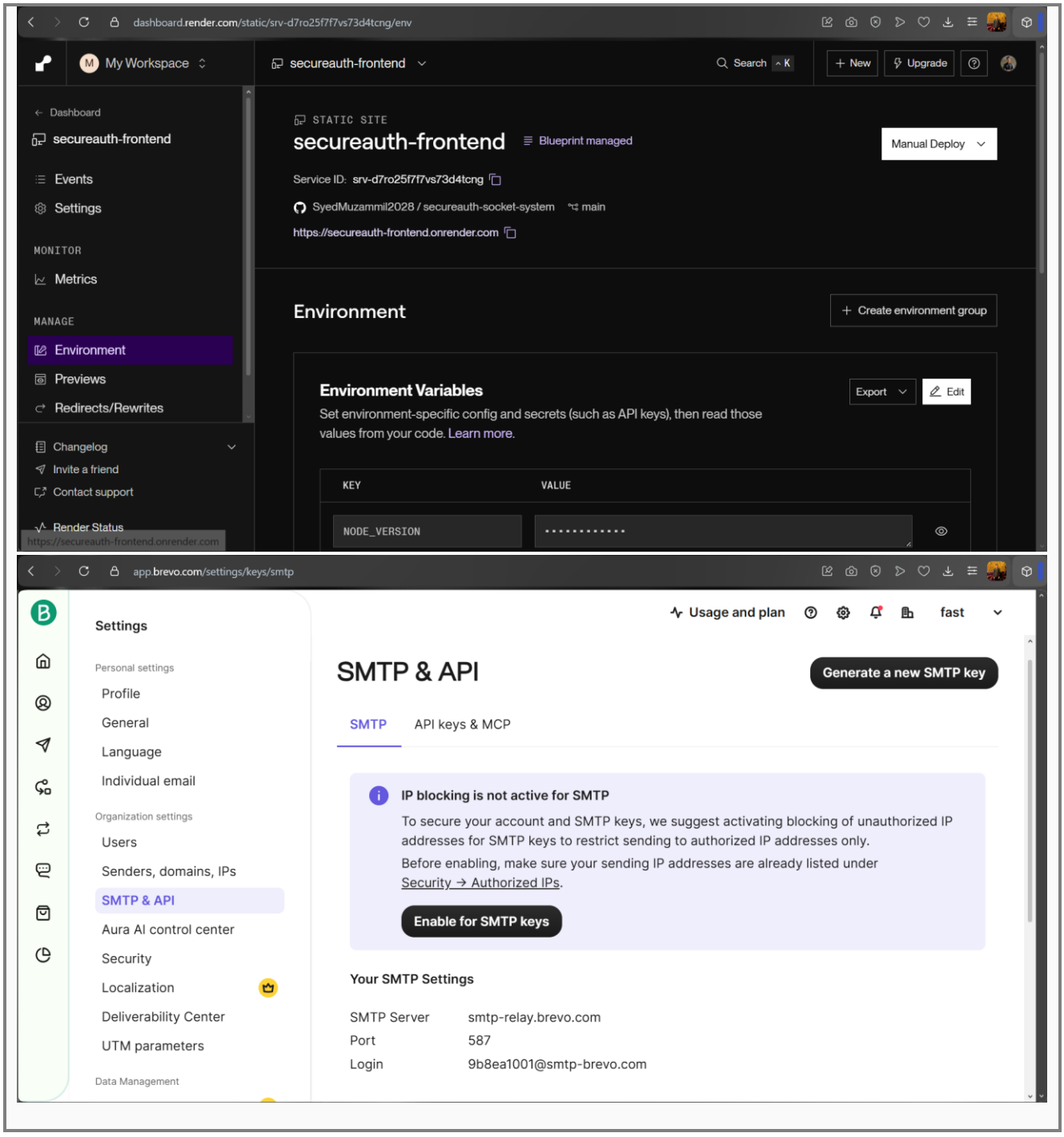
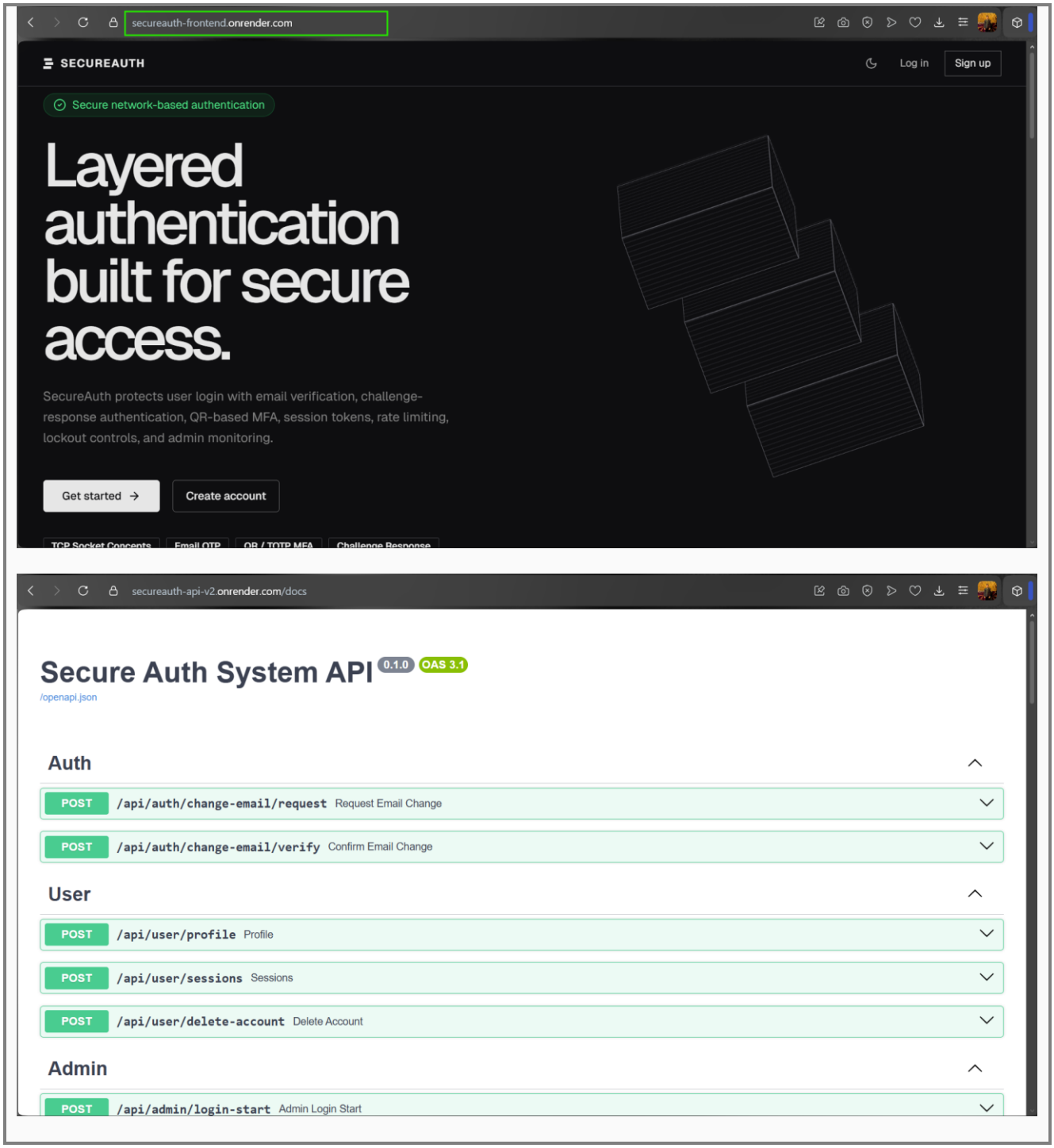


Figure 13. Render Deployment




dashboard.render.com/static/srv-d7ro25f7f7vs73d4tcng/env
My Workspace
secureauth-frontend
Search K
New
Upgrade
Dashboard
secureauth-frontend
Events
Settings
MONITOR
Metrics
MANAGE
Environment
Previews
Redirects/Rewrites
Changelog
Invite a friend
Contact support
Render Status
STATIC SITE
secureauth-frontend
Blueprint managed
Manual Deploy
Service ID: srv-d7ro25f7f7vs73d4tcng
SyedMuzammil2028 / secureauth-socket-system
main
https://secureauth-frontend.onrender.com
Environment
Create environment group
Environment Variables
Set environment-specific config and secrets (such as API keys), then read those values from your code. Learn more.
Export
Edit
KEY
VALUE
NODE_VERSION
app.brevo.com/settings/keys/smtp
Usage and plan
fast
Settings
Personal settings
Profile
General
Language
Individual email
Organization settings
Users
Senders, domains, IPs
SMTP & API
Aura AI control center
Security
Localization
Deliverability Center
UTM parameters
Data Management
SMTP & API
Generate a new SMTP key
SMTP
API keys & MCP
IP blocking is not active for SMTP
To secure your account and SMTP keys, we suggest activating blocking of unauthorized IP addresses for SMTP keys to restrict sending to authorized IP addresses only.
Before enabling, make sure your sending IP addresses are already listed under Security → Authorized IPs.
Enable for SMTP keys
Your SMTP Settings
SMTP Server smtp-relay.brevo.com
Port 587
Login 9b8ea1001@smtp-brevo.com


secureauth-frontend.onrender.com
SECUREAUTH
Log in
Sign up
Secure network-based authentication
Layered authentication built for secure access.
SecureAuth protects user login with email verification, challenge-response authentication, QR-based MFA, session tokens, rate limiting, lockout controls, and admin monitoring.
Get started →
Create account
TCP Socket Concepts
Email OTP
QR / TOTP MFA
Challenge Response
secureauth-api-v2.onrender.com/docs
Secure Auth System API
0.1.0
OAS 3.1
/openapi.json
Auth
POST /api/auth/change-email/request Request Email Change
POST /api/auth/change-email/verify Confirm Email Change
User
POST /api/user/profile Profile
POST /api/user/sessions Sessions
POST /api/user/delete-account Delete Account
Admin
POST /api/admin/login-start Admin Login Start

17. Conclusion

SecureAuth successfully implements a secure network-based authentication system with layered defenses. The project demonstrates socket programming, structured network protocol design, secure credential handling, replay attack prevention, multi-factor authentication, session management, administrative monitoring, and deployment readiness. It fulfills the original proposal scope and extends it with a modern FastAPI backend, React frontend, QR/TOTP MFA, admin user management, user dashboard, Render deployment support, and documentation diagrams.

18. References

9. Kurose, J. F., and Ross, K. W. Computer Networking: A Top-Down Approach.
10. RFC 2104 - HMAC: Keyed-Hashing for Message Authentication.
11. RFC 6238 - TOTP: Time-Based One-Time Password Algorithm.
12. Python Documentation - Socket Programming.
13. FastAPI Documentation.
14. OWASP Authentication Security Guidelines.
15. bcrypt and pyotp library documentation.